Caprice performance: revision 0.3

Affine lengths, fused captures, prefix factoring and compilation reuse | Anonymous author

1.75x faster Boolean matching and **1.63x faster capture-output calls** than v0.2, geometrically across 44 synthetic workloads. Boolean medians improve on 39/44 workloads and beat PCRE2 10.49 on 36/44. These are measured point estimates; compilation and generic expressions remain important gaps.

The original 11 workloads

All 11 v0.3 medians are below PCRE2 direct-JIT medians. Geometric-mean gain over v0.2: 1.19x. Both versions and PCRE2 reuse match storage.

Workload	v0.2 JIT ns/match	v0.3 JIT ns/match	PCRE2 ns/match	Gain over v0.2
Literal	15.5	15.3	33.4	1.01x
Identifier	37.7	33.8	47.4	1.12x
Key/value fields	43.3	30.5	37.4	1.42x
Alternation	19.8	19.7	29.3	1.00x
Short backreference	29.3	23.1	42.2	1.27x
Long backreference	119.5	116.9	243.2	1.02x
Static parameter	24.2	18.0	33.1	1.34x
Lambda guard	37.8	27.5	36.6	1.37x
Dynamic interpolation	34.3	30.2	49.0	1.14x
Length guard	39.2	26.8	36.6	1.46x
Constant lambda	27.7	26.3	35.6	1.05x

Times pool 27 samples from three fresh processes. Engines interleave in randomized order inside each process. Ratios compare workload medians and are then combined geometrically; they are not application-weighted throughput or significance claims.

The next 17 development workloads

1.49x geometric-mean gain over v0.2; lower medians than PCRE2 on 15/17 workloads. These workloads guided this revision.

Workload	v0.2 JIT ns/match	v0.3 JIT ns/match	PCRE2 ns/match	Gain over v0.2
16-byte literal	16.2	16.1	27.1	1.01x
128-byte literal	22.3	22.4	29.6	0.99x
512-byte literal	24.2	24.8	37.9	0.98x
Overlapping alternatives	35.4	19.2	30.1	1.85x
Disjoint alternatives	32.7	33.2	37.9	0.99x
Eight captures	71.4	45.9	53.2	1.55x
Backref without delimiter	172.3	24.0	56.1	7.17x
Long backref, mixed failures	110.6	103.6	145.3	1.07x
Lazy repeat with delimiter	50.4	38.8	50.9	1.30x
Repeated capture	66.0	57.0	37.9	1.16x
Long capture guard	95.7	24.1	69.8	3.98x
Interpolation, varied lengths	52.8	47.9	61.9	1.10x
Wide byte class	93.1	93.2	204.2	1.00x
Negated byte class	92.4	72.6	59.7	1.27x
1 KiB class scan	653.9	638.9	740.2	1.02x
Length guard, larger captures	113.3	26.7	82.7	4.24x
Captures across alternatives	32.7	30.2	33.5	1.09x

The largest previous gap, delimiter-free backreferences, falls from 172.3 ns to 24.0 ns. Long guards improve 3.98x and larger length-guard subjects improve 4.24x. Repeated captures remain 1.51x PCRE2 latency; negated classes remain 1.22x.

The historical ID class_scan_4096 contains 1,024-1,280 byte subjects, including the appended failure byte. The table labels its actual scale.

Fresh validation: 16 workloads

2.71x faster than v0.2 geometrically; 10/16 lower medians than PCRE2. The engine was frozen before seeing these timings.

Workload	v0.2 JIT ns/match	v0.3 JIT ns/match	PCRE2 ns/match	Gain over v0.2
Three copies of a capture	1,046.7	47.7	335.6	21.96x
Affine length with affixes	1,011.9	41.2	221.7	24.53x
Nested capture	45.2	50.0	53.7	0.90x
Two unknown capture lengths	278.9	240.2	140.2	1.16x
Fixed and variable captures	558.6	42.8	180.5	13.05x
Strict length lambda	48.6	28.1	43.8	1.73x
ASCII-folded equality guard	70.9	24.6	64.0	2.88x
Interpolation with affixes	43.6	39.8	57.5	1.10x
Shared-prefix tree	72.9	22.8	29.7	3.20x
Capture barriers in prefixes	61.4	57.8	33.9	1.06x
Repeated captures, 1-16	193.8	157.2	69.6	1.23x
Empty repeated capture	107.1	87.9	44.9	1.22x
Negated class, control bytes	303.1	160.5	244.6	1.89x
Class scan through 8 KiB	2,001.4	1,995.9	1,959.0	1.00x
Static parameter with backref	890.6	41.5	246.8	21.44x
Generic compound guard	444.6	438.8	36.2	1.01x

The generic compound guard still takes 438.8 ns versus PCRE2 plus its unique-capture host predicate at 36.2 ns (12.11x). Nested captures regress 10.6% relative to v0.2. These results are retained without retuning the engine against this corpus.

Why the transformations are valid

One unknown capture length

For a conservative straight-line fragment, the compiler derives N = d + k*l, where N is subject length, d is fixed length, and l is the sole unknown capture length. Completed captures, backreferences and supported length-preserving emissions contribute affine forms. With k > 0, a full match requires N >= d, zero remainder, and the unique length l = (N-d)/k. The original class and repetition bounds are then checked at that endpoint.

Alternation, a second unknown capture length, complex repetition and unsupported expressions stop this analysis. Selected length/equality guards give necessary bounds, while the original guard remains. Early rejection is conditioned on enough expression-byte budget; smaller budgets still execute the strict downstream guard.

Captures and ordered alternatives

An explicit repr(C) start/end slot represents missing, open and completed captures using MAX sentinels. Native code uses Rust-computed offsets, with an initialized, fixed-length slot vector. The public capture-output API materializes Option spans after success; the Boolean API uses slots directly. Generic expressions receive a refreshed public view.

SaveStart; AtomicRun; SaveEnd fuse only when no edge enters their interior. A successful fused scan records one undo entry and commits the completed span; a failed scan leaves it unchanged. Native stores are used only when the program cannot have pending continuations. Shared leading literals factor only across contiguous alternatives; captures and guards are barriers, and alternative order is preserved.

Checked arithmetic and remaining obligations

AffineLengths.lean verifies uniqueness, quotient recovery, divisibility, and sound rejection by remainder or upper bound in Lean 4.19.0. Its five theorems use only propext among reported axioms. This checks arithmetic consequences of an extracted equation, not Rust extraction, Cranelift, or complete regex semantics. Twenty Rust tests check the implementation and retain the earlier exhaustive comparisons.

In the behavioral-presheaf semantics, these rewrites are intended to stay in the same fiber of ordered successful capture observations after forgetting finite resource annotations. Slot conversion is a representation correspondence; fusion and pruning refine traces. Costs remain directed. The proof and testing boundaries are explicit in formal/Revision3.md.

API costs and compilation reuse

Boolean and capture-producing calls were timed separately. The latter calls run_with, checks status and exposes the full capture vector to the benchmark. The gain therefore includes the cost of converting native slots into public Option spans.

Suite	Boolean gain	Capture-output gain
Original 11	1.19x	1.08x
Development 17	1.49x	1.39x
Fresh 16	2.71x	2.56x

Bounded process-local LRU cache

EngineCache keys immutable engines by exact pattern source, the complete static parameter map and mode. Rc ownership keeps a program and native module alive through cache eviction or clearing. The configured capacity bounds retained entries, while callers can retain evicted engines. The cache is local to one thread; lookup is linear in resident entries.

Operation	Median range across 44 workloads
Fresh v0.3 JIT construction	219.3-1,764.5 microseconds
Fresh PCRE2 JIT construction	2.8-25.6 microseconds
v0.3 cache-hit lookup	18.1-26.5 nanoseconds

Fresh JIT construction is 17.9% faster than v0.2 geometrically. A hit avoids construction; it does not reduce miss cost. Hit measurements use capacity 16 with one resident key, include Rc return/drop, and exclude matching. They do not establish realistic multi-key cache throughput.

Allocation measurements

After a complete corpus warmup, 43/44 Boolean workloads allocate no Rust heap objects during matching. The generic compound guard makes 2,032 allocations across its 256 calls. All 44 measured cache-hit paths allocate zero Rust heap objects. These are separately instrumented counts, outside the timing campaign.

Reproduction and next targets

The locked comparison binary links v0.2 and v0.3 as separate Cargo dependency aliases and calls the same PCRE2 10.49 shared library. All eight modes interleave per randomized sample: both versions in PE, Boolean JIT and capture-output JIT modes, plus ordinary and direct PCRE2 JIT calls.

Three fresh processes per corpus; 9 timing samples and 7 construction samples per engine/workload/process. Samples target 20 ms after warmup. Each workload has 256 fixed inputs. The campaign contains 9,504 match rows, 7,392 construction rows and 1,188 cache-hit rows. Acceptance and shared-fragment capture spans are validated before timing; every timed count is checked.

Host: AMD EPYC 9V74, Linux x86-64, logical CPU 0, Rust 1.99.1 release and Cranelift 0.125.3. PCRE2 10.49 uses the previously checked official source build with JIT enabled and -O3 -DNDEBUG. Host load/frequency are uncontrolled. Source, comparison binary, corpus and result hashes accompany the raw data.

Priority work after this revision

Compile the generic typed expression language to a residual IR that avoids temporary strings, argument vectors and environment maps while retaining strict errors and lambda semantics.

Reduce continuation and undo costs for repeated captures and alternatives with capture barriers. Generalize affine reasoning to multiple unknown lengths with ordered feasible endpoints.

Choose PE or native scanners using measured pattern shapes, then add tiering for actual reuse. Cold JIT misses still cost hundreds of microseconds despite cheaper construction and explicit caching.

Scope: full-subject byte semantics and ASCII transforms. PCRE2 host predicates are timed only on uniquely determined full captures. Synthetic cohorts do not establish application performance. New optimizations can change finite exhaustion across versions; Budget remains distinct from rejection, and scans are not charged per byte.

PCRE2 10.49 release

PCRE2 JIT and direct API

PCRE2 performance

Lean theorem proving reference